

FDSproject

June 27, 2026

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

data = pd.read_csv(r"C:\Users\User\Downloads\Bank Loan Status\bankloan(1).csv")
```

```
[2]: data.head(5)
```

```
[2]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001002	Male	No	0.0	Graduate	No	
1	LP001003	Male	Yes	1.0	Graduate	No	
2	LP001005	Male	Yes	0.0	Graduate	Yes	
3	LP001006	Male	Yes	0.0	Not Graduate	No	
4	LP001008	Male	No	0.0	Graduate	No	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	5849	0.0	NaN	360.0	
1	4583	1508.0	128.0	360.0	
2	3000	0.0	66.0	360.0	
3	2583	2358.0	120.0	360.0	
4	6000	0.0	141.0	360.0	

	Credit_History	Property_Area	Loan_Status
0	1.0	Urban	Y
1	1.0	Rural	N
2	1.0	Urban	Y
3	1.0	Urban	Y
4	1.0	Urban	Y

```
[4]: obj = (data.dtypes == 'object')
print("Categorical variables:", len(list(obj[obj].index)))
```

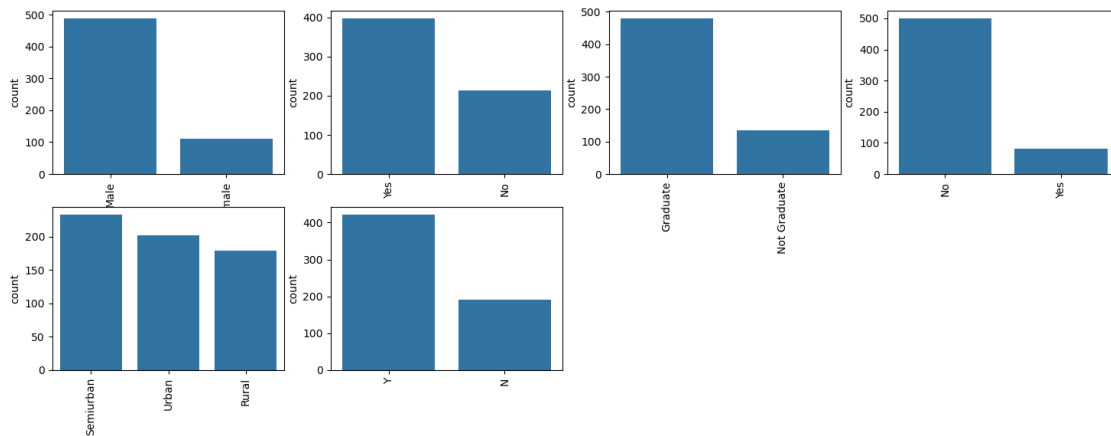
Categorical variables: 7

```
[5]: # Dropping Loan_ID column
data.drop(['Loan_ID'], axis=1, inplace=True)
```

```
[6]: import matplotlib.pyplot as plt
import seaborn as sns

obj = (data.dtypes == 'object')
object_cols = list(obj[obj].index)
plt.figure(figsize=(18, 36))
index = 1

for col in object_cols:
    y = data[col].value_counts()
    plt.subplot(11, 4, index)
    plt.xticks(rotation=90)
    sns.barplot(x=list(y.index), y=y)
    index += 1
```



```
[7]: # Import label encoder
from sklearn import preprocessing

# label_encoder object knows how
# to understand word labels.
label_encoder = preprocessing.LabelEncoder()
obj = (data.dtypes == 'object')
for col in list(obj[obj].index):
    data[col] = label_encoder.fit_transform(data[col])
```

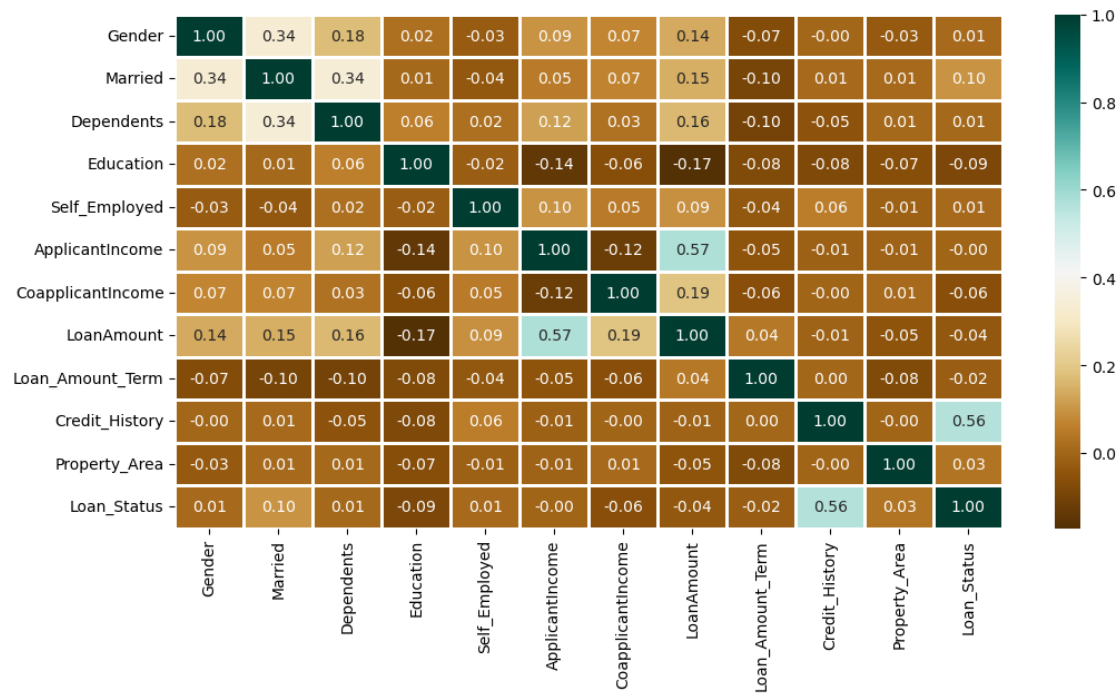
```
[8]: # To find the number of columns with
# datatype==object
obj = (data.dtypes == 'object')
print("Categorical variables:", len(list(obj[obj].index)))
```

Categorical variables: 0

```
[9]: plt.figure(figsize=(12,6))

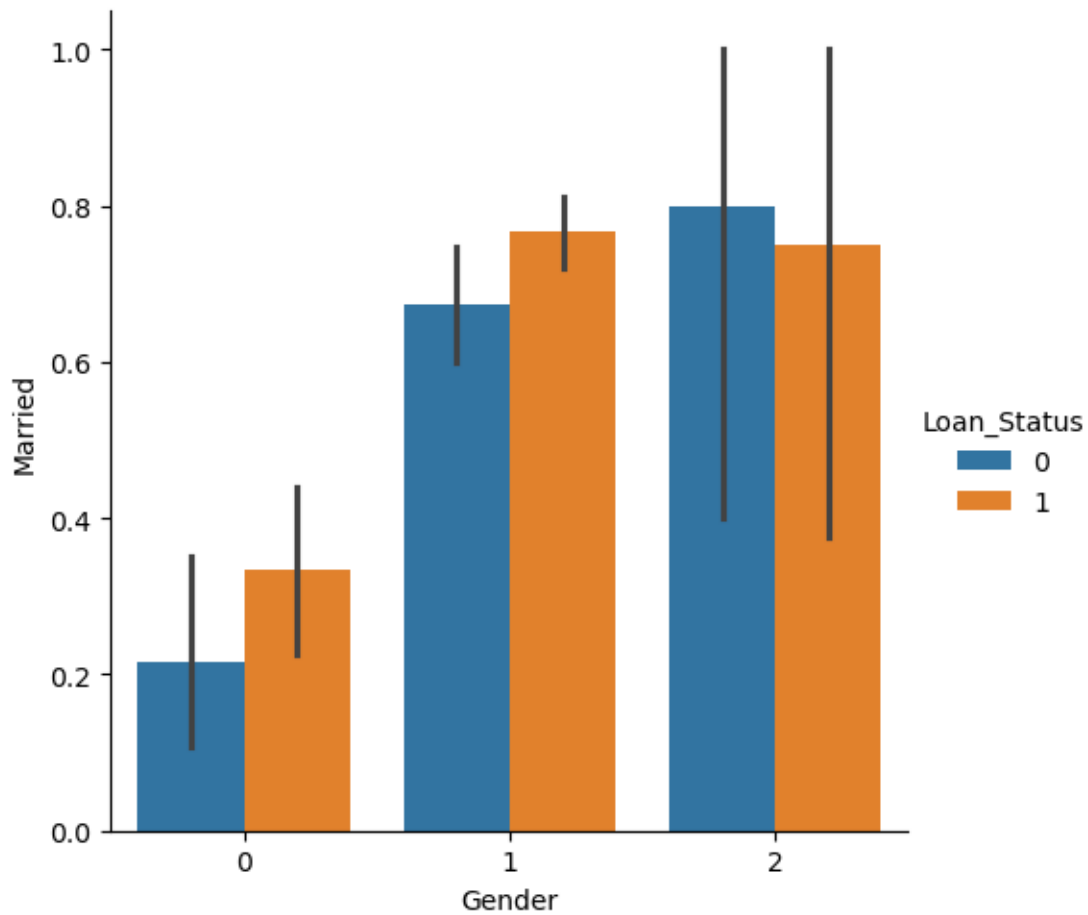
sns.heatmap(data.corr(),cmap='BrBG',fmt='.2f',
            linewidths=2,annot=True)
```

[9]: <Axes: >



```
[10]: sns.catplot(x="Gender", y="Married",
                  hue="Loan_Status",
                  kind="bar",
                  data=data)
```

[10]: <seaborn.axisgrid.FacetGrid at 0x28a7595cad0>



```
[11]: for col in data.columns:
        data[col] = data[col].fillna(data[col].mean())

data.isna().sum()
```

```
[11]: Gender      0
      Married      0
      Dependents   0
      Education     0
      Self_Employed 0
      ApplicantIncome 0
      CoapplicantIncome 0
      LoanAmount     0
      Loan_Amount_Term 0
      Credit_History 0
      Property_Area   0
      Loan_Status     0
      dtype: int64
```

```
[12]: from sklearn.model_selection import train_test_split

X = data.drop(['Loan_Status'],axis=1)
Y = data['Loan_Status']
X.shape,Y.shape

X_train, X_test, Y_train, Y_test = train_test_split(X, Y,

↪4,

X_train.shape, X_test.shape, Y_train.shape, Y_test.shape
```

```
[12]: ((368, 11), (246, 11), (368,), (246,))
```

```
[14]: from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn import metrics

rfc = RandomForestClassifier()
knn = KNeighborsClassifier()
svc = SVC()
lc = LogisticRegression()
for clf in (rfc, knn, svc, lc):
    clf.fit(X_train, Y_train)
    Y_pred = clf.predict(X_test)
    print("Accuracy score of ", clf.__class__.__name__, "=", 100 * metrics.
↪accuracy_score(Y_test, Y_pred))
```

```
Accuracy score of RandomForestClassifier = 78.45528455284553
```

```
Accuracy score of KNeighborsClassifier = 63.82113821138211
```

```
Accuracy score of SVC = 67.07317073170732
```

```
Accuracy score of LogisticRegression = 78.45528455284553
```

```
C:\Users\User\anaconda3\Lib\site-packages\sklearn\linear_model\_logistic.py:473:
```

```
ConvergenceWarning: lbfgs failed to converge after 100 iteration(s) (status=1):
```

```
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT
```

Increase the number of iterations to improve the convergence (max_iter=100).

You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

```
[16]: # making predictions on the testing set
for clf in (rfc, knn, svc, lc):
    clf.fit(X_train, Y_train)
    Y_pred = clf.predict(X_test)
    print("Accuracy score of ",
          clf.__class__.__name__, "=",
          100*metrics.accuracy_score(Y_test,
                                     Y_pred))
```

Accuracy score of RandomForestClassifier = 76.42276422764228

Accuracy score of KNeighborsClassifier = 63.82113821138211

Accuracy score of SVC = 67.07317073170732

Accuracy score of LogisticRegression = 78.45528455284553

C:\Users\User\anaconda3\Lib\site-packages\sklearn\linear_model_logistic.py:473:

ConvergenceWarning: lbfgs failed to converge after 100 iteration(s) (status=1):

STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).

You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

n_iter_i = _check_optimize_result(